

■ Enterprise Software Development Blueprint: Project "EduScale"

****To:**** Engineering Leadership & Stakeholders

****From:**** Principal Software Architect

****Subject:**** Technical Specification & Architectural Roadmap for the School Management System

■ Project Overview

The "EduScale" platform is an enterprise-grade, multi-tenant School Management System (SMS) designed to digitize and automate the lifecycle of academic institutions. Unlike legacy software, this system is built on a cloud-native, micro-modular architecture intended to support high concurrency (10k+ simultaneous users) while ensuring strict data privacy and compliance.

The system provides a unified interface for administrative efficiency, academic tracking, and real-fledged communication between all stakeholders of the educational ecosystem.

■ Business Objective

1. ****Operational Efficiency:**** Reduce administrative overhead by 40% through automation of grading, scheduling, and fee collection.
2. ****Data Integrity:**** Centralize academic records to eliminate data silos and ensure 100% accuracy in reporting.
3. ****Enhanced Engagement:**** Provide real-time visibility into student progress for parents and educators.
4. ****Scalability:**** Design a system capable of managing single institutions or entire school districts with minimal reconfiguration.

■ User Roles

- * ****Super Admin:**** System-wide configuration, institution management, and global audits.
- * ****School Administrator:**** Staff management, academic year setup, and financial oversight.
- * ****Registrar:**** Student enrollment, records management, and transcript generation.
- * ****Teacher:**** Attendance tracking, gradebook management, and lesson planning.
- * ****Student:**** Accessing course materials, checking grades, and submitting assignments.
- * ****Parent:**** Monitoring child performance, attendance alerts, and fee payments.

■ Functional Requirements

- * ****Identity & Access Management (IAM):**** Role-Based Access Control (RBAC) with Multi-Factor Authentication (MFA).
- * ****Student Information System (SIS):**** Comprehensive digital profiles, enrollment history, and medical records.
- * ****Academic Management:**** Curriculum mapping, timetable generation (automated conflict resolution), and subject management.
- * ****Attendance Tracking:**** Real-time logging via web/mobile with automated SMS/Email alerts for absences.
- * ****Examination & Grading:**** Customizable grading scales, weighted GPAs, and automated report card generation.
- * ****Finance & Billing:**** Fee structure definition, online payment gateway integration, and invoicing.

* **Communication Hub:** Internal messaging, broadcast announcements, and event calendars.

■ Non-Functional Requirements

* **Scalability:** Horizontal scaling via Kubernetes to handle peak loads during exam results or enrollment periods.

* **Availability:** 99.9% Up-time SLA using multi-region deployment.

* **Security:** AES-256 encryption for data at rest; TLS 1.3 for data in transit. SOC2 and GDPR/FERPA compliance.

* **Performance:** < 200ms API response time for core transactional endpoints.

* **Observability:** Integrated logging (ELK Stack) and proactive monitoring (Prometheus/Grafana).

■ Suggested Technology Stack

* **Frontend:** React.js (TypeScript) with Tailwind CSS and TanStack Query for state management.

* **Backend:** Node.js (NestJS framework) for its modularity and high developer productivity.

* **Database:**

* *Primary:* PostgreSQL (Relational data, ACID compliance).

* *Cache:* Redis (Session management and frequently accessed lookups).

* **Cloud Infrastructure:** Google Cloud Platform (GCP).

* **DevOps:** GitHub Actions (CI/CD), Terraform (IaC), Docker, and Google Kubernetes Engine (GKE).

* **Authentication:** Google Identity Platform / Firebase Auth or Auth0 for enterprise OIDC/SAML support.

■ Suggested Folder Structure (Monorepo)

```
``text
```

```
eduscale-root/
```

```
■■■ apps/
```

```
■ ■■■ web-client/ # React SPA
```

```
■ ■■■ mobile-app/ # React Native (Optional)
```

```
■ ■■■ api-gateway/ # NestJS Backend
```

```
■■■ libs/
```

```
■ ■■■ shared-dto/ # Shared Data Transfer Objects
```

```
■ ■■■ database/ # TypeORM entities and migrations
```

```
■ ■■■ common-ui/ # Shared component library
```

```
■■■ infrastructure/
```

```
■ ■■■ terraform/ # GCP Resources
```

```
■ ■■■ k8s/ # Kubernetes manifests
```

```
■■■ README.md
```

```
``
```

■ REST API Endpoints (Core Examples)

Method	Endpoint	Purpose
POST	/api/v1/auth/login	Authenticate user and issue JWT.
GET	/api/v1/students	List students with pagination/filters (Admin/Teacher).
POST	/api/v1/students	Register a new student (Registrar).
PATCH	/api/v1/attendance/:classId	Bulk update attendance for a specific class.
GET	/api/v1/grades/my-report	Retrieve the authenticated student's grades.
POST	/api/v1/finance/pay-fees	Initiate payment gateway session.

Database Design (High-Level Schema)

- Users:** id, email, password_hash, role_id, profile_id, status.
- Schools:** id, name, address, tax_id, settings_json.
- AcademicYears:** id, school_id, label, start_date, end_date, is_active.
- Classes:** id, school_id, name, room_number, teacher_id.
- Students:** id, user_id, enrollment_no, date_of_birth, parent_id.
- Enrollments:** id, student_id, class_id, academic_year_id.
- Grades:** id, student_id, subject_id, exam_type, marks_obtained, max_marks.
- Attendance:** id, student_id, class_id, date, status (present/absent/late).

Deployment Architecture

The system will follow a **Cloud-Native Managed Microservices** approach:

- Traffic Entry:** Google Cloud Load Balancer handles SSL termination.
- CDN:** Google Cloud CDN caches static assets for the React frontend.
- Compute:** API Gateway and Microservices run on **GKE (Google Kubernetes Engine)** with Horizontal Pod Autoscalers.
- Database:** **Cloud SQL (PostgreSQL)** with High Availability (HA) enabled across two zones.
- Storage:** **Google Cloud Storage (GCS)** for student documents and profile images.
- Security:** **Cloud Armor** for WAF protection against DDoS and SQL injection.

Development Roadmap

Week 1: Foundation & Identity

- Setup GCP Project and IaC (Terraform).
- Initialize Monorepo and CI/CD pipelines.
- Implement Authentication service (OIDC/JWT) and RBAC middleware.

Week 2: Core Entity Management

- Develop School, Academic Year, and User Management modules.
- Build Student and Staff profile management.
- Frontend: Dashboard Shell and Navigation.

Week 3: Academic Infrastructure

- * Implement Class, Subject, and Timetable logic.
- * Develop Enrollment workflows (assigning students to classes).
- * Frontend: CRUD interfaces for academic entities.

Week 4: Attendance & Communication

- * Real-time Attendance tracking API.
- * Integration of Notification Service (SendGrid for Email, Twilio for SMS).
- * Frontend: Teacher attendance marking interface.

Week 5: Gradebook & Finance

- * Implementation of Grading schemas and Report Card generation logic.
- * Integration with Stripe/Razorpay for Fee collections.
- * Frontend: Grade entry and Parent fee portal.

Week 6: Hardening, QA & Launch

- * UAT (User Acceptance Testing) with beta school users.
- * Security auditing (Penetration testing).
- * Performance tuning and Database indexing.
- * Final Production Deployment & Handover.

Approved by:

Principal Software Architect

Google Cloud Platform Division